



UNIVERSIDAD DE GRANADA

Tecnologías Web
Curso 25/26

Plataforma de Reservas para Centro Deportivo “Sports Center”

Carlos Hoyo Liddle
David Martin Cerezo
Leandro Mazuecos Martín
Isabel Morro Tabares

Índice

1. Descripción del proyecto	4
1.1. Tipos de Usuarios	4
1.2. Requisitos	4
1.2.1. Funcionales	4
1.2.2. No funcionales	4
1.3. Diseño de la interfaz	5
1.3.1. Identidad visual	5
1.3.2. Paleta de colores	5
1.3.3. Tipografía y componentes	5
1.3.4. Licencia y Recursos	6
2. Arquitectura de la aplicación: MVC	6
2.1. Modelos	6
2.2. Controladores	7
2.3. Vistas	7
3. Base de datos	8
4. Enrutamiento de la aplicación	10
4.1. web.php	10
4.2. Esquema de funcionamiento de web.php	11
5. Seguridad Implementada	11
6. Funcionalidades implementadas	12
6.1. Vista pública	12
6.1.1. Inicio	12
6.1.2. Catálogo	13
6.1.3. Instalaciones	14
6.2. Área de usuario	15
6.2.1. Login y registro	15
6.2.2. Reservas	16
6.2.3. Sesión	17
6.3. Área de administración	17
6.3.1. CRUD de actividades e instalaciones	17
6.3.2. Edición de reservas y horarios	19
6.3.3. Gestión de ocupación	20
6.3.4. Restricciones por rol	21
6.4. Diseño responsive	22
6.5. Validaciones	23
6.6. Accesibilidad de la web	23
7. Alojamiento en servidor	24
7.1. Infraestructura	24
7.2. Arquitectura de red: Cloudflare Tunnel	25

7.3. Despliegue del proyecto Laravel	25
7.4. Servicio de sistema y arranque automático	26
7.5. Procedimiento de actualización	26

Enlace al repositorio GitHub: <https://github.com/isamorro/TW/tree/main/miProyecto>

1. Descripción del proyecto

El proyecto consiste en desarrollar una aplicación web para un centro deportivo que permita a los usuarios consultar actividades, horarios e instalaciones disponibles, así como realizar reservas de pistas o clases. La plataforma también incluirá una zona de administración para gestionar actividades, turnos y ocupación.

1.1. Tipos de Usuarios

- **Usuario no identificado:** Puede consultar la página de inicio, el catálogo de actividades, los horarios y la información general del centro.
- **Usuario normal:** Puede iniciar sesión, realizar reservas, consultar su historial y acceder a su área privada.
- **Administrador:** Puede añadir, modificar o eliminar actividades, controlar la ocupación, cancelar turnos y gestionar las reservas.

1.2. Requisitos

1.2.1. Funcionales

- **RF 1.** Registrar e iniciar sesión de usuarios.
- **RF 2.** Mostrar el nombre y tipo de usuario cuando haya iniciado sesión.
- **RF 3.** Consultar actividades e instalaciones disponibles.
- **RF 4.** Ver el detalle de cada actividad.
- **RF 5.** Realizar reservas de pistas o sesiones.
- **RF 6.** Consultar el historial de reservas del usuario.
- **RF 7.** Cancelar reservas, si se permite.
- **RF 8.** Gestionar actividades desde el panel de administración.
- **RF 9.** Controlar el número máximo de plazas por actividad.
- **RF 10.** Evitar reservas con solapamientos horarios.

1.2.2. No funcionales

La aplicación deberá tener un diseño adaptable para ordenador, tableta y móvil. También deberá ser accesible, coherente con la temática deportiva y mantener una separación correcta entre HTML, CSS y PHP.

Todos los formularios estarán hechos en HTML5, con campos obligatorios y validación adecuada. Además, todas las páginas compartirán una cabecera y un pie de página común.

1.3. Diseño de la interfaz

La interfaz de Sports Center sigue una estética deportiva, limpia y moderna, construida sobre Tailwind CSS con un diseño totalmente responsive (móvil, tableta y escritorio). Se prioriza la jerarquía visual, la legibilidad y la consistencia entre vistas, manteniendo cabecera y pie comunes en toda la aplicación.

1.3.1. Identidad visual



SportsCenter

Isotipo (mancuerna sobre forma orgánica) y logotipo en versión horizontal.

1.3.2. Paleta de colores

Color	Nombre	HEX	Uso principal
	Verde de marca	#589337	Botones principales, enlaces destacados y acentos.
	Verde hover	#4A7D2F	Estado hover y activo del color de marca.
	Verde claro	#ECFDF5	Chips, badges y bloques de apoyo.
	Carbón principal	#111827	Texto principal, footer y fondos oscuros.
	Carbón secundario	#4B5563	Texto secundario y descripciones.
	Gris fondo	#F9FAFB	Fondo general de secciones.
	Blanco	#FFFFFF	Tarjetas, cabecera y zonas de contenido.
	Gris borde	#D1D5DB	Bordes, separadores y líneas finas.

1.3.3. Tipografía y componentes

Se utiliza la familia sans-serif del sistema para garantizar legibilidad y rendimiento. Los componentes se construyen con clases utilitarias de Tailwind (tarjetas, botones, formularios, tablas y modales) siguiendo un patrón consistente: bordes suaves, esquinas redondeadas y estados hover y focus claramente diferenciados para mejorar la accesibilidad.

1.3.4. Licencia y Recursos

Para la utilización de imágenes hemos utilizado principalmente dos fuentes principales:

1. <https://unsplash.com/es/licencia> Unsplash plataforma web con gran cantidad de recursos gráficos e imágenes profesionales con licencia libre para estudiantes y proyectos no comerciales, es decir, uso indicado en nuestro caso.
2. Imágenes generadas mediante inteligencia artificial, este método nos ha dado grandes posibilidades para la generación de imágenes a nuestro gusto. Los LLM utilizados para este propósito han sido los siguientes: [Nanobanan](#) y [ChatGPT](#).

Con estas fuentes hemos obtenido un resultado profesional y moderno, a la altura de páginas web comerciales.

2. Arquitectura de la aplicación: MVC

El sistema se basa en el patrón MVC de Laravel y utiliza Tailwind CSS para el diseño.

2.1. Modelos

- Activity.php: Modelo encargado de representar las actividades deportivas disponibles en la plataforma, almacenando información como nombre, descripción, capacidad máxima e imagen asociada.
- Installation.php: Modelo utilizado para gestionar las instalaciones deportivas del centro, incluyendo su descripción, estado de disponibilidad e imagen representativa.
- User.php: Modelo encargado de representar a los usuarios registrados de la aplicación, gestionando la autenticación, el almacenamiento seguro de contraseñas, las notificaciones y el control de acceso según el tipo de usuario.
- Reservation.php: Modelo encargado de representar las reservas realizadas por los usuarios, gestionando la relación entre el usuario que reserva y la sesión reservada, así como su estado (activa, cancelada o completada).
- SessionActivity.php: Modelo encargado de representar las sesiones programadas de cada actividad, almacenando fecha, hora de inicio y fin, e indicando en qué instalación se desarrolla. Es el elemento sobre el que los usuarios realizan sus reservas y donde se aplican los controles de capacidad y solapamiento.

2.2. Controladores

- ActivityController.php: Controlador encargado de la gestión completa de actividades deportivas, permitiendo al administrador crear, modificar, eliminar y validar actividades almacenadas en la base de datos.
- AdminController: Controlador encargado de gestionar las funcionalidades propias del administrador dentro de la aplicación. Centraliza el acceso a las secciones de administración y permite organizar la gestión de actividades, instalaciones, sesiones y reservas desde el panel administrativo.
- InstalationController.php: Controlador responsable de administrar las instalaciones deportivas del centro, incluyendo operaciones CRUD, validación de datos y control del estado de disponibilidad.
- LoginController.php: Controlador encargado de gestionar el proceso de autenticación de usuarios, incluyendo inicio y cierre de sesión mediante el sistema Auth de Laravel.
- RegisterController.php: Controlador utilizado para gestionar el registro de nuevos usuarios, validando los datos introducidos y almacenando las contraseñas de forma segura mediante cifrado.
- ReservationController.php: Controlador encargado de gestionar el flujo completo de reservas por parte del usuario normal. Permite listar el historial de reservas del usuario autenticado, mostrar el formulario de confirmación de una sesión concreta, almacenar nuevas reservas aplicando las validaciones de capacidad (RF9) y solapamiento horario (RF10), y cancelar reservas tanto propias del usuario como, en el caso del administrador, ajenas.
- SessionActivityController: Controlador encargado de administrar los horarios o sesiones disponibles para las actividades deportivas. Permite crear, modificar y eliminar sesiones asociadas a una actividad e instalación concreta, validando la fecha, la franja horaria y la relación con los datos existentes en la base de datos.

2.3. Vistas

- admin/
 - activities/
 - create.blade.php: Formulario para que el administrador cree nuevas actividades.
 - edit.blade.php: Formulario para modificar actividades existentes.
 - installations/
 - create.blade.php: Formulario para crear nuevas instalaciones deportivas.
 - edit.blade.php: Formulario para editar instalaciones existentes.
 - reservations/
 - history.blade.php: Vista del panel de administración que muestra el historial completo de reservas registradas en el sistema.
 - index.blade.php: Vista para consultar y cancelar reservas activas.

- sessions/
 - [create.blade.php](#): Formulario para crear nuevas sesiones de actividades.
 - [edit.blade.php](#): Formulario para modificar sesiones existentes.
- layouts/
 - [app.blade.php](#): Plantilla principal compartida por todas las páginas, con cabecera, menú, contenido y pie de página.
- login/
 - [form-errors.blade.php](#): Bloque que muestra los mensajes de error de validación en una lista, destacándose con estilos en color rojo.
 - [layout.blade.php](#): Pantalla base con estilos Tailwind.
 - [login.blade.php](#): Formulario de acceso.
 - [profile.blade.php](#): Vista del perfil de usuario que muestra sus datos, acceso según su rol, ajustes de cuenta y cierre de sesión.
 - [register.blade.php](#): Formulario de registro.
- [actividad-detalle.blade.php](#): Página de detalle de una actividad concreta.
- [catalogo.blade.php](#): Vista pública del catálogo dinámico de actividades.
- [contacto.blade.php](#): Página con información de contacto y datos del desarrollador.
- [home.blade.php](#): Página principal del sitio web.
- [instalaciones.blade.php](#): Vista pública de instalaciones deportivas disponibles.
- [reservas.blade.php](#): Página del área de usuario para gestionar o consultar reservas.
- [reserva-crear.blade.php](#): Vista del formulario de confirmación de reserva. Muestra los detalles de la sesión seleccionada.

3. Base de datos

3.1. Tabla *users*

Campo	Tipo	Descripción
id	Bigint	Identificador único del usuario
name	String	Nombre del usuario
email	String	Correo electrónico único
email_verified_at	Timestamp	Fecha de verificación del email
password	String	Contraseña cifrada
role	Enum(user, admin)	Tipo de usuario
remember_token	String	Token para mantener la sesión iniciada

created_at	Timestamp	Fecha de creación
updated_at	Timestamp	Fecha de actualización

3.2. Tabla *activities*

Campo	Tipo	Descripción
id	Bigint	Identificador de actividad
name	String	Nombre de la actividad
description	Text	Descripción de la actividad
capacity	Interger	Número máximo de plazas
image_path	String	Ruta de la imagen
created_at	Timestamp	Fecha de creación
updated_at	Timestamp	Fecha de actualización

3.3. Tabla *installations*

Campo	Tipo	Descripción
id	Bigint	Identificador de instalación
name	String	Nombre de la instalación
description	Text	Descripción de la instalación
image_path	String	Ruta de la imagen
status	enum(available, maintenance)	Estado de disponibilidad
created_at	Timestamp	Fecha de creación
updated_at	Timestamp	Fecha de actualización

3.4. Tabla *sessions_activities*

Campo	Tipo	Descripción
id	Bigint	Identificador de sesión

activity_id	Foreignid	Actividad asociada
installation_id	Foreignid	Instalación asociada
date	date	Fecha de la sesión
start_time	Time	Hora de inicio
end_time	Time	Hora de finalización
created_at	Timestamp	Fecha de creación
updated_at	Timestamp	Fecha de actualización

3.5. Tabla *reservations*

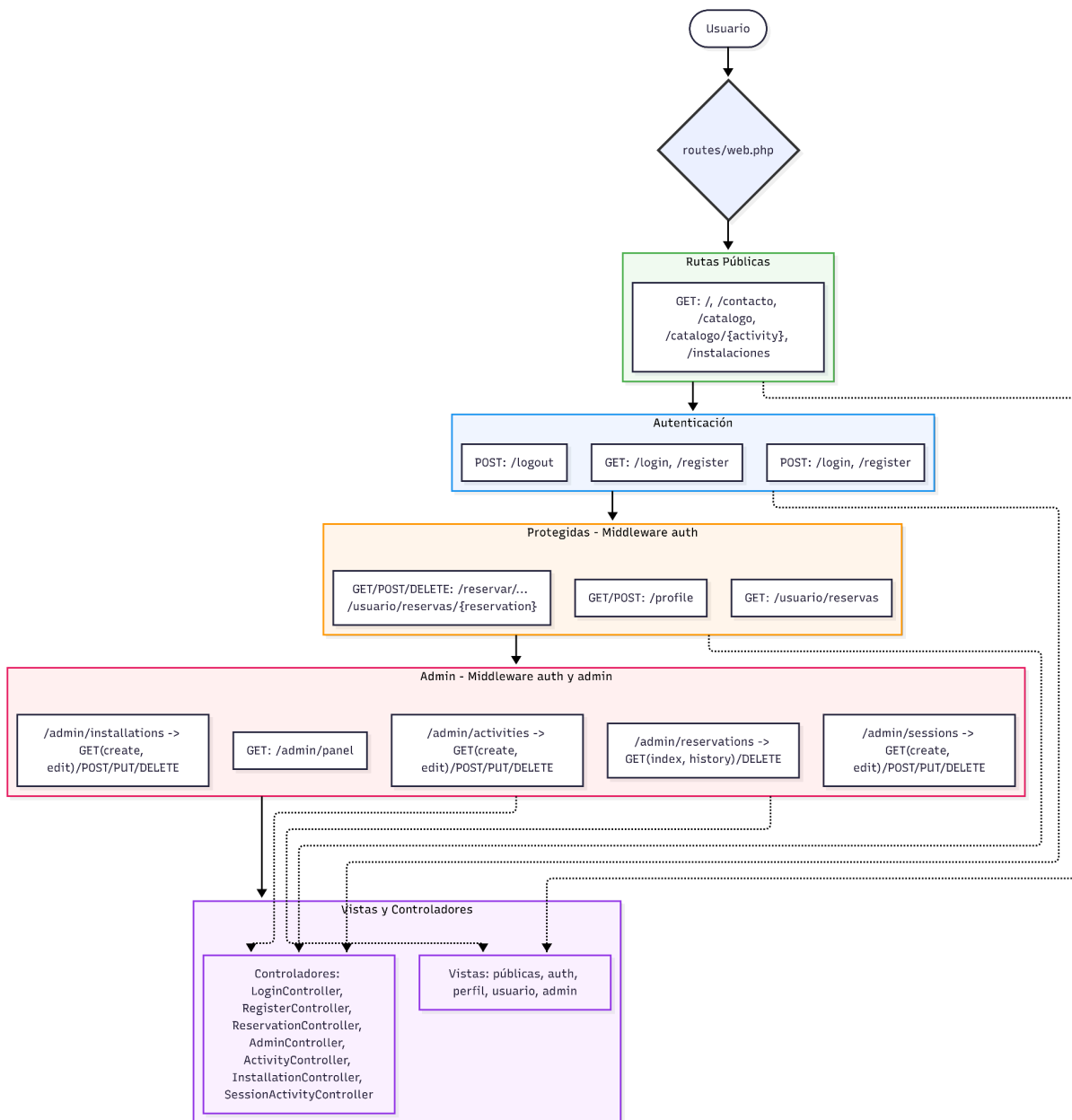
Campo	Tipo	Descripción
id	Bigint	Identificador de reserva
user_id	Foreignid	Usuario que realiza la reserva
session_id	Foreignid	Sesión reservada
status	enum(active, cancelled, completed)	Estado de la reserva
created_at	Timestamp	Fecha de creación
updated_at	Timestamp	Fecha de actualización

4. Enrutamiento de la aplicación

4.1. web.php

El enrutamiento de nuestra página web se ha desarrollado en web.php donde se han definido las rutas especificadas en la práctica y se han aplicado los middleware necesarios.

4.2. Esquema de funcionamiento de web.php



5. Seguridad Implementada

- **Protección CSRF:** Todos los formularios incluyen `@csrf` para evitar ataques de falsificación de petición en sitios cruzados.
- **Encriptación:** Las contraseñas nunca se guardan en texto plano, siempre se usa el algoritmo Bcrypt.
- **Validación:** Se validan los datos de entrada tanto en formato como en existencia.

- **Limitación de peticiones:** Se permiten 5 peticiones por minuto por API:
`"->middleware('throttle:5,1')"`

6. Funcionalidades implementadas

6.1. Vista pública

6.1.1. Inicio

La página de inicio es una de las partes más importantes del desarrollo web enfocado en captar clientes o motivarlos a usar el servicio por eso se ha diseñado con un punto de vista dirigido a el registro en el club, con botones grandes y simples y con unas secciones accesibles para el usuario nuevo, para diseñar se ha seguido el patrón de página como <https://www.fitnesspark.es>, <https://synergym.es>, <https://www.anytimefitness.es>. De esta manera la plataforma ofrece una experiencia de usuario a la altura de un servicio comercial.

La estructura de la página de inicio es la siguiente:

1. Sección Hero con botón que al hacer clic mediante un link `<a href="#clubs">` redirige al usuario a la sección de selección de ciudades.
2. Carrusel "Nuestros Espacios" ([Swiper.js](#)): Esta sección construye un carrusel con las instalaciones dinámicamente iterando las disponibles en la base de datos
 - 2.1. La ruta principal en web.php construye la vista y consulta modelos: Consulta para el home: `Installation::where('status','available')->orderBy('name')->get()` y pasa `compact('installations')` a la vista.
 - 2.2. Resultado: la vista recibe `$installations` y las renderiza en el carrusel.
3. Sección “Nuestros clubes”: Sección estática imitando las secciones comunes en paginas de centros deportivos, esta simplemente redirige mediante un link a la ruta de registro al hacer click `<a href="{{ route('register') }}">`
4. Sección “App”: Sección estática con la implementación de un estilo visual avanzado mediante tailwind, mediante la utilización de sus características se muestran las posibilidades de una buena utilización de la tecnología.
5. Sección “Newsletter”: Sección obligatoria en una página profesional, normalmente orientada a la captación de clientes mediante servicios de correo electrónico, en nuestro caso se ha implementado pero con un botón que envía al usuario a la zona de registro. `<form action="/register"> <button type="submit">`

6.1.2. Catálogo

El catálogo de actividades permite a los usuarios consultar de forma visual todas las actividades deportivas disponibles en el centro.

Los datos se obtienen dinámicamente de la siguiente forma:

1. Las actividades se almacenan en la tabla *activities* de la base de datos y son gestionadas mediante el modelo *Activity*.
2. Desde el controlador se recuperan todas las actividades y se envían a la vista:
`$activities = Activity::orderBy('name')->get();`
3. Posteriormente, la vista *catalogo.blade.php* genera automáticamente una tarjeta para cada actividad mediante un bucle Blade:
`@forelse($activities as $activity)`

De esta forma, cualquier nueva actividad añadida desde el panel de administración aparecerá automáticamente en el catálogo sin necesidad de modificar el código HTML.

Por otro lado, cada actividad se representa mediante una tarjeta visual compuesta por:

- imagen representativa,
- nombre de la actividad,
- descripción,
- número de plazas disponibles,
- enlace a la página de detalle.

Las imágenes se cargan dinámicamente utilizando `asset($activity->image_path)`, permitiendo así almacenar únicamente la ruta de la imagen en la base de datos.

Además, el catálogo se ha desarrollado utilizando *CSS Grid* mediante *TailwindCSS* con `grid` `grid-cols-1 sm:grid-cols-2 lg:grid-cols-3`. Luego, el sistema adapta automáticamente el número de columnas según el tamaño de pantalla:

- móviles: 1 columna,
- tabletas: 2 columnas,
- pantallas grandes: 3 columnas.

Finalmente, se ha incorporado un menú lateral de navegación que muestra el listado completo de actividades disponibles. En la versión de escritorio, este menú permanece fijado en la pantalla mediante la clase *sticky top-28*, de forma que acompaña al usuario mientras realiza scroll por la página. Esto mejora la navegación, ya que permite cambiar rápidamente entre actividades sin tener que volver a la parte superior. En dispositivos móviles, este menú se adapta a un formato desplegable para ocupar menos espacio y mantener una interfaz más cómoda.

6.1.3. Instalaciones

La sección de instalaciones permite a los usuarios consultar todas las zonas deportivas disponibles en el centro, mostrando información detallada sobre su estado y disponibilidad.

Las instalaciones se gestionan dinámicamente mediante la base de datos utilizando el modelo *Installation*.

Los datos se obtienen de la siguiente forma:

1. Las instalaciones se almacenan en la tabla *installations* y son gestionadas mediante el modelo *Installation*.
2. Desde la ruta correspondiente se recuperan todas las instalaciones ordenadas alfabéticamente mediante `$installations = Installation::orderBy('name')->get();`
3. Posteriormente, la vista *instalaciones.blade.php* genera automáticamente una tarjeta visual para cada instalación mediante un bucle Blade con `@forelse($installations as $installation)`

De esta forma, cualquier nueva instalación añadida desde el panel de administración aparecerá automáticamente en la página pública sin necesidad de modificar el código HTML.

Cada instalación se representa mediante una tarjeta compuesta por:

- imagen representativa,
- nombre de la instalación,
- descripción,
- estado de disponibilidad.

Las imágenes se cargan dinámicamente utilizando `asset($installation->image_path)`, permitiendo almacenar únicamente la ruta de la imagen en la base de datos.

Además, el sistema muestra visualmente el estado de cada instalación, que es disponible o en mantenimiento. Esto se implementa mediante etiquetas dinámicas de color `@if($installation->status === 'available')`

La interfaz se ha desarrollado utilizando CSS Grid mediante TailwindCSS con `grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3`, adaptando automáticamente el número de columnas según el tamaño de pantalla:

- móviles: 1 columna,
- tabletas: 2 columnas,
- pantallas grandes: 3 columnas.

6.2. Área de usuario

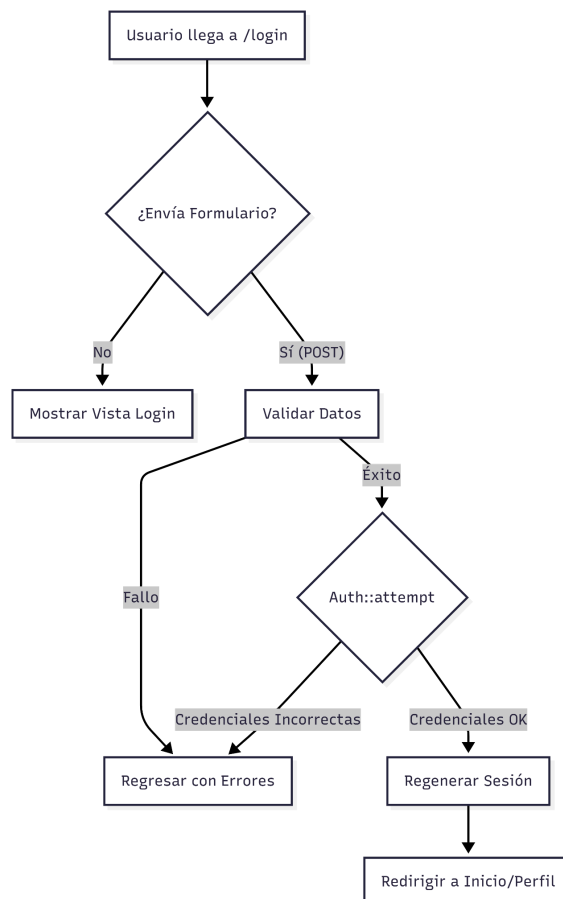
6.2.1. Login y registro

El proceso de inicio de sesión (login) es el siguiente:

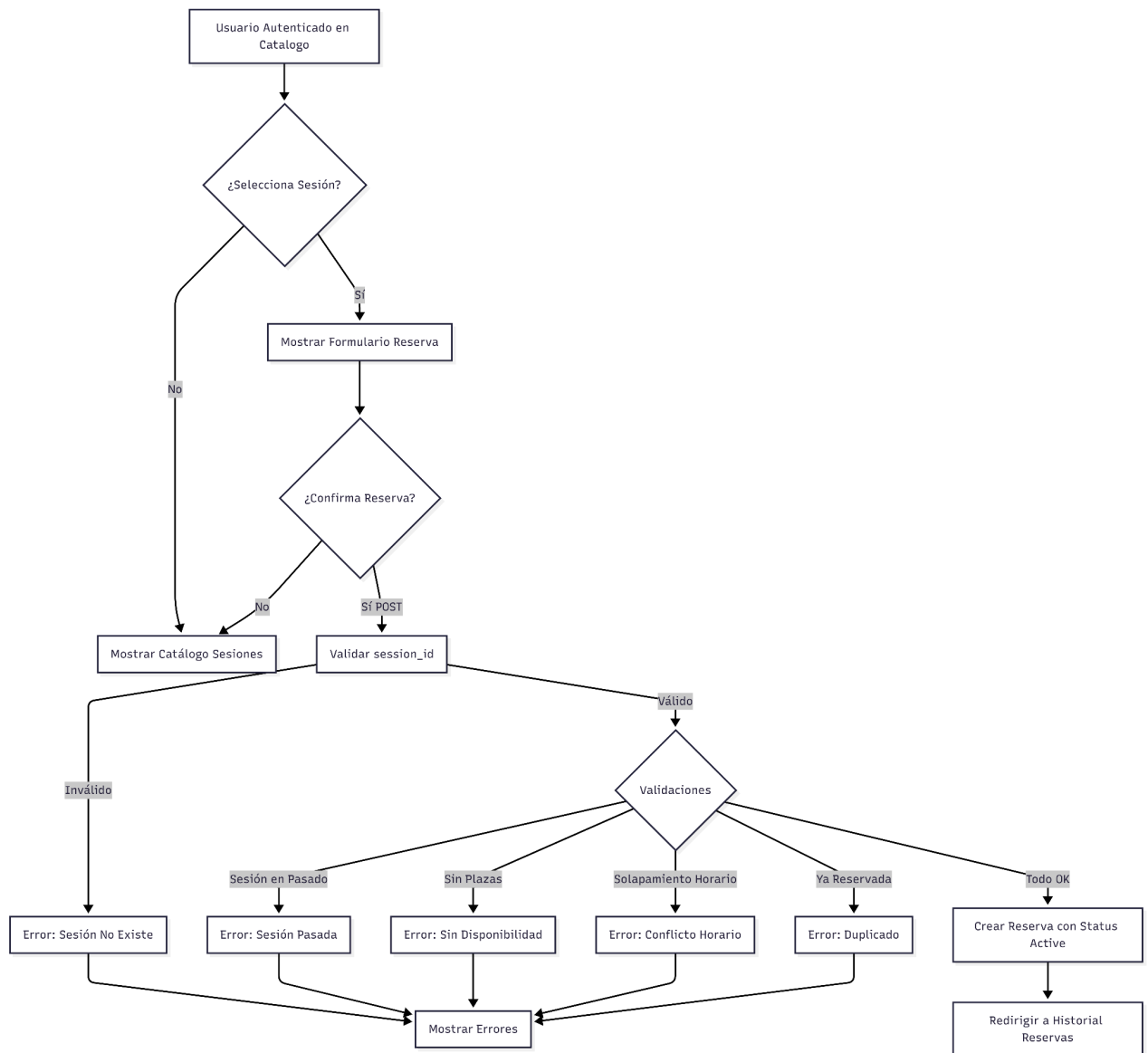
1. El usuario accede a */login*.
2. Introduce su email y password.
3. El *LoginController* valida que los campos no estén vacíos.
4. Se utiliza *Auth::attempt()* para verificar las credenciales contra la base de datos.
5. Si es correcto, se regenera la sesión para evitar ataques de fijación de sesión y se redirige al usuario.
6. Si es incorrecto, se devuelve a la vista con un mensaje de error.

Por otra parte, el proceso de registro de un usuario es:

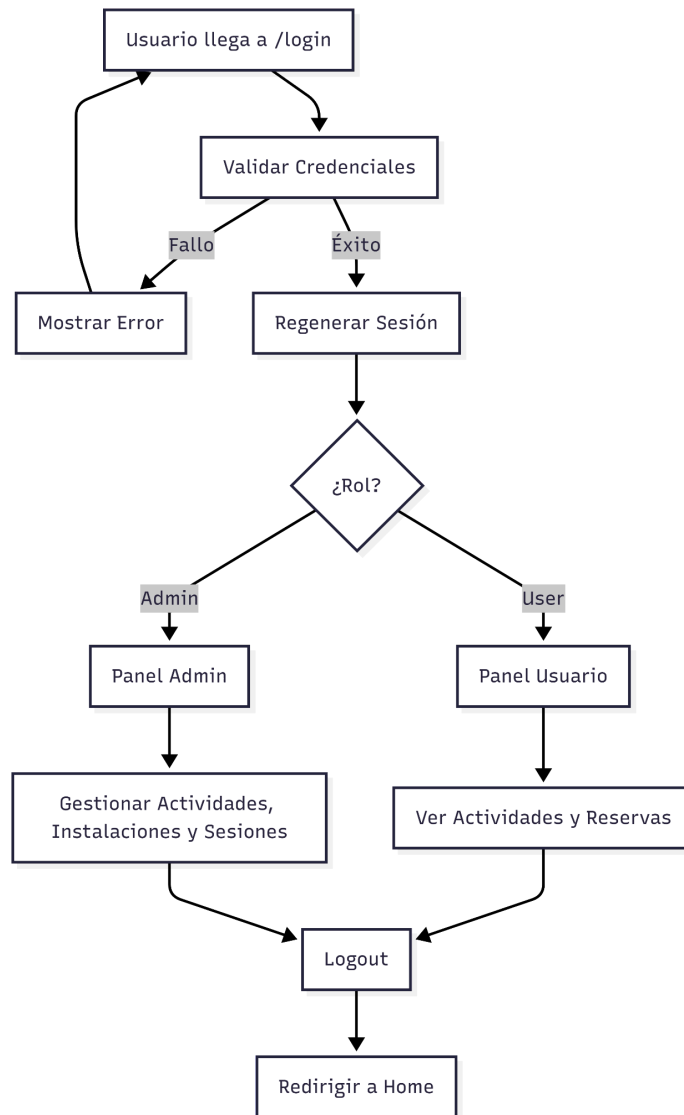
1. El usuario accede a */register*.
2. Introduce nombre, email, password y confirmación.
3. El *RegisterController* valida que el email sea único y la contraseña coincida.
4. Se crea el registro en la tabla users con la contraseña encriptada mediante *Hash::make()*.
5. El sistema inicia sesión automáticamente al nuevo usuario y lo redirige.



6.2.2. Reservas



6.2.3. Sesión



6.3. Área de administración

6.3.1. CRUD de actividades e instalaciones

El área de administración permite gestionar de forma dinámica tanto las actividades como las instalaciones del centro deportivo. Para ello se ha implementado un CRUD completo, es decir, operaciones de creación, lectura, actualización y eliminación de registros. Estas funcionalidades solo están disponibles para usuarios autenticados con rol *admin*.

Control de acceso

Las rutas de administración están protegidas mediante middleware:

```
Route::middleware(['auth', 'admin'])->group(function () {

    Route::get('/admin/panel', [AdminController::class, 'index'])->name('admin.panel');

    Route::get('/admin/activities', [ActivityController::class,
'adminList'])->name('admin.activities.index');

    Route::resource('/admin/activities', ActivityController::class)
        ->except(['index'])
        ->names('admin.activities');

    Route::resource('/admin/installations', InstallationController::class)

        ->names('admin.installations');

    Route::delete('/admin/reservations/{reservation}', [ReservationController::class,
'destroy'])

        ->name('admin.reservations.destroy');
});
```

De esta forma, un usuario no identificado o un usuario normal no puede acceder a las páginas de administración.

CRUD de actividades

El CRUD de actividades permite al administrador crear, editar y eliminar actividades deportivas. El controlador utilizado es *Admin/ActivityController.php*, y realiza las siguientes operaciones:

- **create()**: Muestra el formulario para crear una nueva actividad
- **store()**: Valida y guarda la nueva actividad en la base de datos
- **edit()**: Muestra el formulario de edición de una actividad
- **update()**: Valida y actualiza los datos de la actividad
- **destroy()**: Elimina una actividad existente

Los formularios permiten introducir el nombre, la descripción, la capacidad, y la ruta de imagen. Por otro lado, la validación se realiza en el controlador:

```
$data = $request->validate([
    'name' => 'required|string|max:255',
    'description' => 'required|string',
```

```
'capacity' => 'required|integer|min:1',
'image_path' => 'nullable|string|max:255',
]);
```

Una vez creada, editada o eliminada una actividad, el usuario vuelve al catálogo.

CRUD de instalaciones

El CRUD de instalaciones permite al administrador gestionar las zonas deportivas del centro. El controlador utilizado es *Admin/InstallationController.php* y permite:

- **create():** Muestra el formulario para crear una nueva instalación
- **store():** Valida y guarda la nueva instalación en la base de datos
- **edit():** Muestra el formulario de edición de una instalación
- **update():** Valida y actualiza los datos de la instalación
- **destroy():** Elimina una instalación existente

Los campos gestionados son el nombre, la descripción, la ruta de imagen, y el estado, que puede ser “available” o “maintenance”. La validación aplicada es:

```
$data = $request->validate([
    'name' => 'required|string|max:255',
    'description' => 'nullable|string',
    'image_path' => 'nullable|string|max:255',
    'status' => 'required|in:available,maintenance',
]);
```

Integración con las vistas públicas

Las páginas *catalogo.blade.php* e *instalaciones.blade.php* muestran botones adicionales cuando el usuario tiene rol de administrador. En ambos aparece el botón para crear nueva actividad o instalación, icono de editar e icono de eliminar. Esto se controla mediante:

```
@auth
@if(auth()->user()->role === 'admin')
```

Así, la interfaz pública se adapta según el tipo de usuario conectado.

6.3.2. Edición de reservas y horarios

El panel de administración permite al usuario con rol admin gestionar tanto las reservas registradas como los horarios programados del centro deportivo, complementando los controles automáticos de capacidad (RF9) y solapamiento horario (RF10) que el motor de reservas aplica sobre las solicitudes de los usuarios normales.

Gestión de reservas

El administrador puede cancelar cualquier reserva activa, independientemente del usuario que la realizara, cumpliendo el RF7 desde la perspectiva administrativa. La operación se ejecuta mediante una petición *HTTP DELETE* sobre la ruta */admin/reservations/{reservation}*, que delega en el método *destroy* del *ReservationController*. Este método verifica previamente que el usuario solicitante tiene rol *admin* antes de permitir la cancelación de reservas ajenas; en caso contrario, devuelve un error *HTTP 403*. La cancelación se efectúa mediante eliminado lógico, actualizando el campo *status* de la reserva al valor *cancelled*, lo que mantiene la trazabilidad histórica.

Gestión de horarios (sesiones programadas)

El administrador dispone de un CRUD completo de sesiones desde el panel de administración. La interfaz permite:

- **Crear** nuevas sesiones mediante el formulario de la vista *admin/sessions/create.blade.php*, especificando la actividad asociada, la instalación donde se imparte, la fecha y las horas de inicio y fin.
- **Modificar** sesiones existentes a través de la vista *admin/sessions/edit.blade.php*.
- **Eliminar** sesiones que ya no vayan a impartirse.

Esta funcionalidad permite al administrador adaptar la oferta de sesiones a las necesidades operativas del centro: añadir clases especiales, modificar franjas horarias estacionales o retirar turnos sin demanda. Los formularios utilizan validación HTML5 (atributos *required* y *type* adecuados para fecha y hora) y se complementan con validación en backend antes de persistir los cambios.

6.3.3. Gestión de ocupación

El panel de administrador incluye una sección dedicada al control de la ocupación y la gestión de los turnos de los usuarios, lo que permite al administrador visualizar en tiempo real el estado de las actividades y cancelar reservas cuando sea necesario.

Cálculo de la ocupación

Para obtener los datos de la ocupación real, se ha implementado una consulta utilizando el constructor de consultas de Laravel (*DB::table*) en el controlador *AdminController.php*. Se realiza un *LeftJoin* entre la tabla de actividades y la de reservas para contar el número de usuarios inscritos y compararlo con la capacidad máxima:

```
$occupancy = DB::table('activities')
->leftJoin('reservations', 'activities.id', '=', 'reservations.activity_id')
->select(
    'activities.name',
```

```

        'activities.capacity',
        DB::raw('count(reservations.id) as current_occupancy')
    )
    ->groupBy('activities.id', 'activities.name', 'activities.capacity')
    ->get();

```

Cancelación de turnos

En la vista *panel-admin.blade.php*, se despliega una tabla HTML con el listado de reservas activas mostrando el usuario, la actividad y la fecha. Para la cancelación de turnos se ha optado por utilizar validación nativa HTML5 sin depender de JavaScript.

Cada fila de la tabla incluye un formulario independiente que apunta a la ruta de borrado administrada por el *ReservationController*:

```

<form action="{{ route('admin.reservations.destroy', $reservation) }}" method="POST">
    @csrf
    @method('DELETE')
    <button type="submit" class="bg-red-100 text-red-700 p-2 rounded-lg">
        Cancelar Turno
    </button>
</form>

```

Al enviar este formulario, el controlador elimina el registro de la base de datos, liberando instantáneamente la plaza para que otro usuario pueda ocuparla.

6.3.4. Restricciones por rol

Para garantizar la seguridad de la aplicación y asegurar que las funciones de gestión sólo sean accesibles por el personal autorizado, se ha implementado un sistema de control de acceso basado en roles (*user* y *admin*).

Protección de rutas (Middleware)

Se ha desarrollado un middleware personalizado llamado *AdminMiddleware.php* que actúa como barrera de seguridad para todas las peticiones dirigidas al panel de control. Su función *handle* verifica dos condiciones: que el usuario haya iniciado sesión y que su rol sea estrictamente el de administrador.

```

public function handle(Request $request, Closure $next): Response
{
    if (Auth::check() && Auth::user()->role === 'admin') {
        return $next($request);
    }
}

```

```

    }

    return redirect('/')->with('error', 'No tienes permisos para acceder a esta sección.');
```

Este middleware se aplica en el archivo de enrutamiento *web.php*, agrupando todas las rutas sensibles (panel, creación, edición y borrado) bajo el bloque *Route::middleware(['auth', 'admin'])->group(function () { ... });*. Esto previene accesos no autorizados incluso si un usuario normal intenta forzar la URL directamente en el navegador.

Protección en las vistas

A nivel de interfaz gráfica, la aplicación se adapta dinámicamente al tipo de usuario conectado. Utilizando las directivas de Blade, se ocultan los elementos de administración (como los botones de modificar actividades, añadir instalaciones o acceder al panel) a los usuarios estándar.

Esto se logra envolviendo el código HTML restringido dentro de sentencias condicionales que comprueban el rol del usuario autenticado:

```

@auth
    @if(auth()->user()->role === 'admin')
        @endif
@endauth
```

De esta manera, se mantiene una separación limpia entre la vista pública y las herramientas de gestión interna, mejorando tanto la seguridad como la experiencia de usuario.

6.4. Diseño responsive

Para facilitar la implementación de un diseño responsive, nos hemos decidido por la utilización de un framework web: Tailwind CSS el cual servirá de base para construir una interfaz atractiva y responsive. Principalmente nos hemos orientado a la utilización de contenedores adaptables centrados en pantalla para facilitar la visualización en cualquier dispositivo.

Para lograr una web responsive hemos aplicado los siguientes estilo a los componentes:

- Viewport
 - `<meta name="viewport" content="width=device-width, initial-scale=1.0">`
- Imágenes:
 - Responsive: *h-8 sm:h-9 md:h-12 w-auto*
 - Ocultas condicionalmente: *hidden sm:block* para no mostrar en teléfono
- Formularios de autenticación
 - Contenedor: *px-4 py-10 sm:px-6 lg:px-8* para padding adaptable
 - Máximo ancho: *max-w-md* contenedor fijo

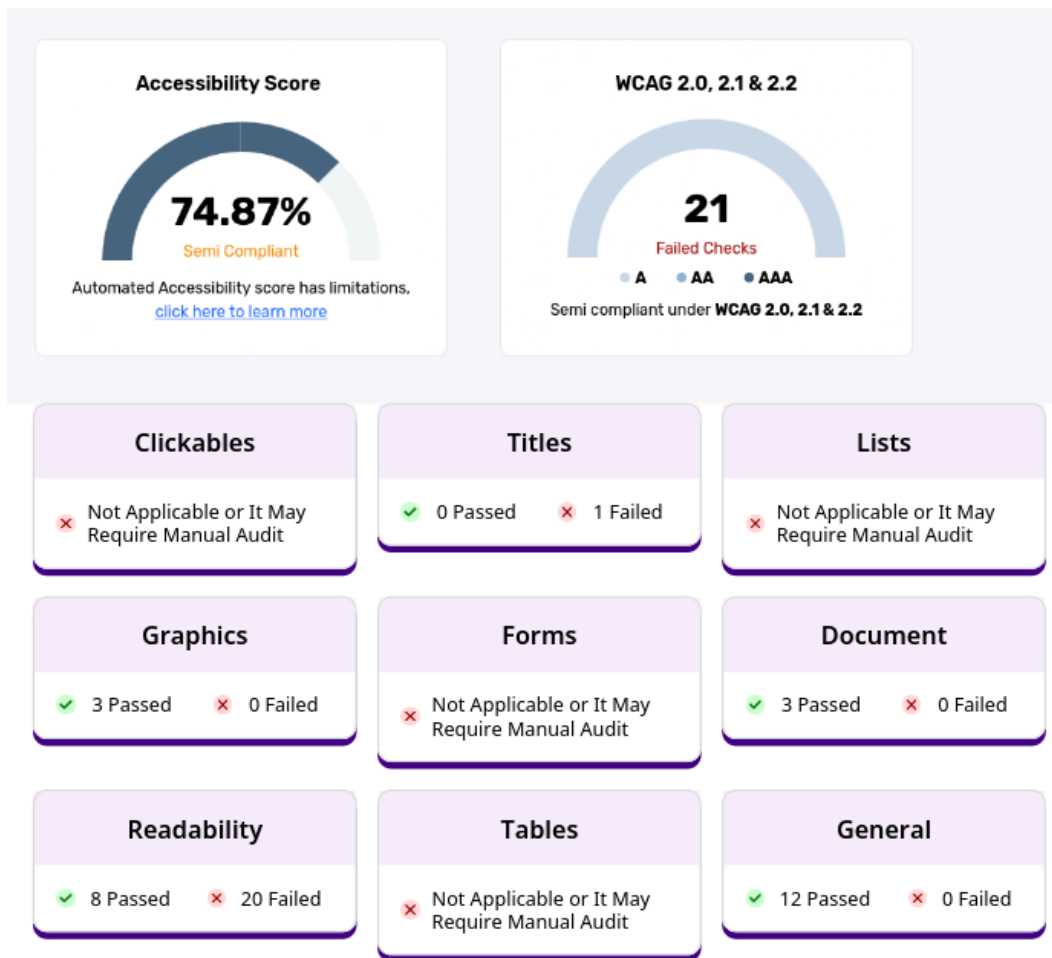
- Hero padding: *p-8 md:p-12* para aumenta en escritorio
- Layout
 - Navegación: *hidden lg:flex* oculta en móvil, visible en escritorio
 - Menú móvil: *hidden lg:hidden* solo en móvil
 - Sidebar: *hidden lg:block lg:col-span-1* oculto en móvil
 - Grid principal: *grid grid-cols-1 lg:grid-cols-4*: 1 columna móvil → 4 columnas escritorio

6.5. Validaciones

- Login
 - Correo se valida con *pattern=[a-z0-9.-]+@[a-z0-9.-]+*
 - *required* para los campos necesarios
 - *maxlength="40"* para evitar longitudes innecesarias en las peticiones
- Registro
 - Correo se valida con *pattern=[a-z0-9.-]+@[a-z0-9.-]+*
 - *required* para los campos necesarios
 - *minlength="2"* para no enviar peticiones vacías
 - *maxlength="40"* para evitar longitudes innecesarias en las peticiones
 - *minlength="8"* para forzar al usuario a contraseñas de alta seguridad
- Perfil
 - *required* para los campos necesarios
 - *minlength="2"* para no enviar peticiones vacías
 - *maxlength="40"* para evitar longitudes innecesarias en las peticiones
 - *minlength="8"* para forzar al usuario a contraseñas de alta seguridad

6.6. Accesibilidad de la web

Hemos revisado la accesibilidad del proyecto SportsCenter utilizando un informe automático de accesibilidad y aplicado varias mejoras para corregir los principales problemas detectados. Entre los cambios realizados se encuentran la mejora de los títulos de las páginas mediante etiquetas `<title>` claras y específicas, la revisión de la jerarquía de encabezados para mantener una estructura coherente, la eliminación o sustitución de enlaces vacíos como `href="#"`, la incorporación de textos alternativos adecuados en imágenes y logotipos, la adición de etiquetas *aria-label* en botones que solo contenían iconos.



7. Alojamiento en servidor

Una vez completado el desarrollo, la aplicación se ha desplegado en un servidor propio para que el sitio esté accesible públicamente bajo el dominio <https://sportcenters.athenorserver.com>. A continuación se describen la infraestructura y el procedimiento empleados.

7.1. Infraestructura

El alojamiento se realiza sobre un equipo con sistema operativo Debian que actúa como servidor permanente en una red doméstica. Se ha optado por esta configuración en lugar de contratar un VPS comercial por motivos de aprendizaje y flexibilidad.

El stack instalado en el servidor está compuesto por:

- **PHP 8.4**, lenguaje en el que se ejecuta Laravel.
- **MariaDB 10.4**, motor de base de datos.
- **Composer**, gestor de dependencias de PHP, utilizado para instalar el framework Laravel y sus paquetes.

- **Node.js** y **pnpm**, gestores empleados para compilar los assets de frontend (Vite, Tailwind, scripts).
- **cloudflared**, cliente oficial de Cloudflare para el establecimiento del túnel inverso.

7.2. Arquitectura de red: Cloudflare Tunnel

Para exponer el servidor a Internet sin necesidad de abrir puertos en el router doméstico, se ha optado por Cloudflare Tunnel. Esta tecnología establece una conexión saliente persistente desde el servidor hacia la red de Cloudflare mediante el cliente *cloudflared*; cuando un usuario solicita *sportcenters.athenorserver.com*, la petición llega primero a la edge de Cloudflare, que la reenvía a través del túnel hasta el servidor doméstico.

Las ventajas de este enfoque, frente a un despliegue tradicional con apertura de puertos, son:

- **No requiere abrir puertos** en el router doméstico, lo que reduce la superficie de ataque del equipo.
- **HTTPS automático**: Cloudflare termina la conexión TLS en su edge y reenvía la petición al servidor como HTTP simple, por lo que el certificado público no necesita renovación manual.
- **Dominio propio**: el subdominio *sportcenters.athenorserver.com* se registra sobre el dominio raíz *athenorserver.com*, adquirido por nosotros, gestionado íntegramente en Cloudflare. El registro DNS se configura automáticamente al crear la ruta pública del túnel desde el panel de Zero Trust.

7.3. Despliegue del proyecto Laravel

El proyecto se ha desplegado siguiendo estos pasos:

1. Clonado del repositorio Git en el servidor.
2. Instalación de las dependencias de PHP mediante `composer install`.
3. Instalación y compilación de los assets de frontend con `pnpm install` y `pnpm run build` (Usábamos npm, pero dado que recientemente han modificado los repositorios y ahora hay una alta posibilidad de instalar malware, lo cambiamos por seguridad)
4. Configuración del fichero `.env` con las credenciales de la base de datos y la URL pública (`APP_URL=https://sportcenters.athenorserver.com`).
5. Creación de la base de datos `miproyecto` y de un usuario dedicado laravel con permisos sobre ella en MariaDB.
6. Ejecución de las migraciones y los seeders con `php artisan migrate:fresh --seed` para construir el esquema y poblarlo con datos de prueba.
7. Asignación de los permisos adecuados a los directorios `storage/` y `bootstrap/cache/`, donde Laravel necesita escribir logs, caché y sesiones.

Para que Laravel reciba correctamente la información del protocolo HTTPS desde Cloudflare y genere URLs internas válidas (login, formularios, redirecciones), se ha configurado el middleware de proxies de confianza en `bootstrap/app.php`: `$middleware->trustProxies(at: '*');`

7.4. Servicio de sistema y arranque automático

Para que la aplicación se mantenga en ejecución sin necesidad de conservar una sesión SSH abierta, y se reinicie automáticamente tras cualquier reinicio del equipo, Laravel se ha registrado como servicio de systemd en `/etc/systemd/system/laravel-sportscenter.service`. El servicio invoca el servidor de desarrollo integrado de Laravel (`php artisan serve --host=127.0.0.1 --port=8000`) y se configura con la directiva `Restart=always` para recuperarse automáticamente en caso de caída.

De forma equivalente, MariaDB y el cliente cloudflared arrancan como servicios del sistema, lo que garantiza que la web vuelve a estar plenamente operativa de manera autónoma tras cualquier reinicio del equipo.

7.5. Procedimiento de actualización

Cuando se incorporan nuevos cambios al repositorio por parte de algún miembro del equipo, el servidor se actualiza ejecutando un script automatizado (`actualizar-web.sh`) que realiza la rutina completa:

1. Detiene el servicio de Laravel.
2. Sincroniza el código local con la rama main del repositorio remoto (`git pull`).
3. Actualiza las dependencias de PHP y de frontend (`composer install`, `pnpm install`, `pnpm run build`).
4. Aplica las migraciones pendientes (`php artisan migrate --force`).
5. Limpia las cachés de configuración, rutas y vistas.
6. Reinicia el servicio de Laravel.